Self Parameter

Self parameter is a reference to the current instance of the class. It allows us to access the attributes and methods of the object.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name} is barking!")

d = Dog("Tommy", 23)
d.bark()
```

NOTE:

```
class Check:
    def __init__(self):
        print("Address of self = ", id(self))

obj = Check()
print("Address of class object = ", id(obj))
```

What is a Constructor?

A constructor is a special function in a class that automatically runs when you create an object.

Think of it like setting up a new phone — when you buy it, you enter your name, Wi-Fi, etc. That setup is the constructor.

Why Use a Constructor?

- To give values to the object at the time of creation.
- It saves time and ensures the object is ready to use.

Python Constructor Syntax

```
class ClassName:
    def __init__(self, arg1, arg2):
        self.arg1 = arg1
        self.arg2 = arg2
```

- __init__ is the constructor in Python.
- self refers to the current object.
- It runs automatically when you create the object.

Points about Constructor

Point	Explanation
Name of constructor	__init__
Auto-run	Runs automatically when object is created
Purpose	To initialize object variables
Use self	To access object's own variables and methods

Destructor in Python

1. What is a Destructor?

- A Destructor is a special method in a class that is automatically called when an object is deleted or destroyed.
- In Python, the destructor method is named:

```
def __del__(self):
    # cleanup code
```

- It is the opposite of the constructor (__init__).
    - Constructor → Initializes object.
    - Destructor → Cleans up resources when the object is no longer needed.

---

2. Why Do We Need Destructor?

- To free resources (like closing a file, releasing memory, disconnecting a database, stopping a network connection, etc.)
- Python uses garbage collection, so destructors are not used often.
- Still, they are useful when working with external resources.

---

3. Simple Example of Destructor

```
class Student:
    def __init__(self, name):
        self.name = name
        print(f"Constructor called -> Object created for {self.name}")

    def __del__(self):
        print(f"Destructor called -> Object destroyed for {self.name}")

# Creating object
s1 = Student("Atharva")

# Deleting object explicitly
del s1

print("Program ended")
```

Output:

Constructor called → Object created for Atharva

Destructor called → Object destroyed for Atharva

Program ended

---

4. Real-Life Example – File Handling

Imagine a situation where we open a file.

- Constructor → Opens the file.
- Destructor → Closes the file automatically when object is deleted.

```
class FileHandler:
    def __init__(self, filename):
        self.file = open(filename, "w")
        print("File opened")
```

```
    def write_data(self, data):
        self.file.write(data)

    def __del__(self):
        self.file.close()
        print("File closed automatically")

# Usage
handler = FileHandler("sample.txt")
handler.write_data("Hello, OOP in Python!")
del handler   # Destructor will be called here
```

---

5. Important Points

1. Destructor is called when:
    - Object goes out of scope.
    - del keyword is used.
    - Python garbage collector destroys the object.
2. Destructor may not run immediately after del, because garbage collection is automatic.
3. Best practice: Use destructors for clean-up tasks.

---

6. Real-Life Analogy

Think of:

- Constructor as checking into a hotel → You get a room, keys, etc.
- Destructor as checking out of the hotel → You return keys, settle bill, and free the room for others.

Methods – Things an Object Can Do

Methods are like actions. If an object is a mobile, the method can be "call" or "click photo".

Syntax:

```
class ClassName:
    def method_name(self):
        print("This is a method")
```

Full Example

```
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def show(self):
        print(f"{self.name} is in grade {self.grade}.")

s1 = Student("Aditya", 12)
s1.show()
```

Concept Summary

Concept	Simple Meaning	Python Keyword	Syntax Example
Class	Design/Plan	class	class Car:
Object	Real thing made from class	(no keyword)	mycar = Car("BMW")
Method	Action it can do	def	def drive(self):

Example

```
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def show_info(self):
        print(f"Name: {self.name}")
        print(f"Grade: {self.grade}")

s1 = Student("Aditya", 12)
s1.show_info()
```

Explanation

Part	What it does
class Student:	Creates a class called Student
__init__	Constructor – sets name and grade
show_info	Method – prints details of the student
s1 = Student(...)	Creates an object
s1.show_info()	Calls the method on the object

Feature	Constructor (__init__)	Destructor (__del__)
Purpose	Initializes object variables	Cleans up resources when object is deleted
When Called	Automatically called when an object is created	Automatically called when an object is destroyed
Keyword/Method	__init__(self, ...)	__del__(self)
Runs	At the start of object's lifecycle	At the end of object's lifecycle
Analogy	Checking into a hotel (getting keys, room setup)	Checking out of a hotel (returning keys, freeing room)
Common Uses	Set initial values, open files, connect to DB	Close files, release memory, disconnect DB
Control	Runs immediately when object is created	May not run immediately after del (garbage collection controlled)
Example	def __init__(self, name): self.name = name	def __del__(self): print("Object destroyed")

Interview Questions.

Company / Year / Source	Question Asked	What They Want to Test / Tips
Amazon	"Define the use of destructor program in Python."	They ask a direct definition and may expect a small code example. Be ready to explain limitations.

Cisco / Capgemini / TCS / IBM / Amazon	“Explain constructor & destructor in Python and their functions.”	Good chance of follow-ups like “when is destructor called?” or “why not rely on destructor for critical cleanup?”
Generic OOP / experienced level interviews	“What are Constructors and Destructors?”	Basic, but expect follow-ups: order of constructor / destructor, multiple destructors, exceptions in __del__, etc.
Verve Copilot Interview Questions	“What do Python class destructors reveal about your understanding of memory management?”	They want deeper understanding: reference counting, garbage collection, cycles, nondeterministic nature of __del__, etc.